

dbt Training — Tier 2 Exercises

Setup: Before starting these exercises, ensure the exercise project runs on your machine.

See `README.md` for setup instructions.

Run `dbt build` from the `excersices/` directory to confirm everything passes before Module 08.

Python version: Python $\leq$ 3.12 is required. dbt-core 1.11 is not compatible with Python 3.13+.

Module 08 — Seeds and Variables

Duration: 45 min

Context: The exercise project has a full staging and Silver layer from Modules 01–07. This module adds two lookup seeds and a configurable variable to an existing Silver model.

The exercise project already contains `seeds/country_codes.csv` and `seeds/product_categories.csv`. You will load them, write a Gold mart that uses a seed, and add a runtime-configurable variable to an existing model.

Task 1 — Load seeds and inspect the output

Run `dbt seed` and confirm that both seeds loaded successfully:

```
dbt seed
```

You should see two `OK created seed` lines in the output — one for `country_codes` and one for `product_categories`.

Then inspect the seed data:

```
dbt show --select country_codes --limit 10
```

Expected output: Rows with three columns — `country_code`, `country_name`, `region`. You should see entries for DE (Germany), AT (Austria), CH (Switzerland), and others.

Answer in one sentence: Why is `country_codes` a seed and not a source?

Verify:

```
dbt ls --select country_codes
```

Expected: `dbt_training.country_codes`

Task 2 — Debug `models/3_gold/mrt_country_summary.sql`

A colleague has submitted `models/3_gold/mrt_country_summary.sql` for review. Run the model and inspect the output:

```
dbt run --select mrt_country_summary
dbt show --select mrt_country_summary --limit 10
```

What do you notice? Look at the `contact_count`, `deal_count`, and `total_deal_amount` columns.

All counts are 0. Something is wrong with the join. Open `models/3_gold/mrt_country_summary.sql` and find the bug.

- Hint
- Expected fix

Verify:

```
dbt run --select mrt_country_summary
dbt show --select mrt_country_summary --limit 5
```

Expected: `contact_count > 0` for DE, FR, GB, IT, DK.

Task 3 — Add a `min_deal_amount` variable to `fct_deal.sql`

Step 1 — Add the variable default to `dbt_project.yml`

The `vars:` key goes at the **root level** of the file — not nested inside `models:` or `seeds:`.

Before:

```
name: 'dbt_training'
version: '1.0.0'

models:
  dbt_training:
    +materialized: view
```

After:

```
name: 'dbt_training'
version: '1.0.0'

vars:
  min_deal_amount: 0

models:
```

```
dbt_training:
  +materialized: view
```

Step 2 — Edit `models/silver/fct_deal.sql`

Find the WHERE clause in `fct_deal.sql` (or add one if missing) and add a filter using the variable:

```
WHERE amount >= {{ var('min_deal_amount', 0) }}
```

Step 3 — Run with the default

```
dbt run --select fct_deal
```

Expected: the model runs and all deals are included (default is 0, so all deals pass the filter).

Step 4 — Override the variable at runtime

```
dbt run --select fct_deal --vars '{"min_deal_amount": 500}'
```

Step 5 — Verify the substitution in compiled SQL

```
dbt compile --select fct_deal --vars '{"min_deal_amount": 500}'
```

Open `target/compiled/dbt_training/models/silver/fct_deal.sql` and confirm the literal `500` appears in the WHERE clause — not `{{ var(...) }}`.

► Expected compiled WHERE clause

Bonus — Find a seed candidate in existing code

Open any two staging or Silver models. Find one place where a lookup value is hardcoded in a CASE WHEN or WHERE clause that would be better managed as a seed CSV.

Write down:

1. Which model contains it (file path)
2. What the seed file would look like (column names + 3 sample rows)
3. How you would rewrite the model to use `{{ ref('your_seed_name') }}`

There is no single right answer — the goal is recognizing the pattern.

Bonus — Rename output tables with `+alias`

The seeds in this project have names like `raw_contacts.csv` but the source tables they simulate are called `contacts`. Open `dbt_project.yml` and find the seed section.

Question: How does dbt know to load the seed as `contacts` instead of `raw_contacts` ?

► Answer

Three practical uses of `+alias` to know:

Use case	Example
Fix a name mismatch	Seed file is <code>raw_contacts.csv</code> but the source expects <code>contacts</code> — alias solves it without renaming the file
Point to a smaller dev table	In development, alias a 1 000-row sample instead of the full 10 M-row table to keep runs fast
Incremental testing	Alias to <code>table_v1</code> , run; alias to <code>table_v2</code> (added/changed rows), run again — verify the incremental model picks up only the delta

Try it: In `dbt_project.yml`, change the `+alias` for `raw_contacts` from `contacts` to `contacts_test`, run `dbt seed --select raw_contacts --target test`, then run `dbt run --select stg_hubspot__contacts --target test`. What error do you get and why?

Change it back when done.

Module 09 — Jinja and Macros

Duration: 90 min

Context: The exercise project has a staging layer and Silver models from earlier modules. You are adding a shared utility macro and applying it in two places.

Before starting, run `dbt deps` to ensure `dbt_utils` is installed.

Task 1 — Write `macros/safe_cast.sql`

Create the file `macros/safe_cast.sql`. The macro must:

- Take three parameters: `column_name`, `target_type`, and `fallback` (optional, default `None`)
- When `fallback` is provided: output `COALESCE(TRY_CAST(column_name AS target_type), fallback)`
- When `fallback` is omitted: output `TRY_CAST(column_name AS target_type)`

Remember: macro definitions use `{% macro %}` / `{% endmacro %}` (statement delimiters), not `{{ }}`.

- Starter skeleton
- Expected solution

Verify:

```
dbt compile --select stg_hubspot__deals
```

If the macro file has a syntax error, dbt will report a parse error here. Fix any errors before moving to Task 2.

Task 2 — Use `safe_cast` in `stg_hubspot__deals.sql`

Confirm the model exists:

```
dbt ls --select stg_hubspot__deals
```

Open `models/staging/hubspot/stg_hubspot__deals.sql`. The `amount` column is currently selected as a raw value. Replace it with a safe cast to `DOUBLE`:

```
{{ safe_cast('amount', 'DOUBLE') }} AS amount,
```

Verify the macro resolves in compiled output:

```
dbt compile --select stg_hubspot__deals
```

Open `target/compiled/dbt_training/models/staging/hubspot/stg_hubspot__deals.sql`. Confirm the `amount` line shows `TRY_CAST(amount AS DOUBLE)` — no Jinja in the output.

► Expected model (amount line)

Run and confirm:

```
dbt build --select stg_hubspot__deals
```

Expected: green output — model builds and all tests pass.

Task 3 — Add `dbt_utils.generate_surrogate_key` to `fct_deal.sql`

Confirm the model exists:

```
dbt ls --select fct_deal
```

Open `models/silver/fct_deal.sql`. Add a surrogate key column `deal_key` by hashing `deal_id`. Place this in the **source CTE**, as the first selected column:

```
{{ dbt_utils.generate_surrogate_key(['deal_id']) }} AS deal_key,
```

► Expected source CTE

Verify:

```
dbt build --select fct_deal
```

Open `target/compiled/.../models/silver/fct_deal.sql`. Confirm `deal_key` compiles to an `MD5(...)` expression — not `{{ dbt_utils.generate_surrogate_key(...) }}`.

Expected compiled key line (DuckDB):

```
MD5(CAST(deal_id AS TEXT)) AS deal_key,
```

Bonus — Macro candidate review

Open any two Silver models (`dim_patient.sql` , `fct_prescription.sql` , or `fct_deal.sql`). Find one CTE or expression that repeats across models and answer:

1. Is it a good candidate for extraction as a macro? Why or why not?
2. If yes: write the macro signature (name and parameters). You don't need to implement it.
3. If no: explain what makes it better left as SQL.

Use the decision framework from the module:

- Repeated **structural pattern** (type coercion, key generation) → macro candidate
- **Business logic** (CASE WHEN rules, domain classifications) → leave in SQL

Module 10 — Slowly Changing Dimensions and Snapshots

Duration: 90 min

Context: `stg_hubspot__contacts` is already built. The snapshot file `snapshots/snap_patients.sql` has been pre-built for you as a scaffold. Your job is to understand the config, run it, and observe SCD2 in action.

Snapshots live in the `snapshots/` folder (not `models/`). They are run with `dbt snapshot` , not `dbt run` .

Task 1 — Review `snapshots/snap_patients.sql`

Open the file `snapshots/snap_patients.sql` . Read it carefully and answer:

1. What is the `unique_key` ?
2. What strategy is used? What does it require from the source data?
3. Which column triggers a new SCD2 row when it changes?
4. Which columns are tracked in the snapshot?

► Answers

Task 2 — Build the initial snapshot

Run `dbt snapshot` to create the snapshot table:

```
# Snowflake:
dbt snapshot
```

```
# DuckDB local testing (run dbt seed first if not already done):
dbt seed --target test
dbt snapshot --target test
```

Expected terminal output: `Created snapshot snapshots.snap_patients`

Verify the snapshot was created:

```
dbt show --select snap_patients --limit 10
```

Or query directly:

```
SELECT COUNT(*) FROM snapshots.snap_patients;
SELECT contact_id, email, dbt_valid_from, dbt_valid_to FROM snapshots.snap_patients LIMIT 5;
```

Expected:

- 20 rows (one per patient/contact)
- All rows have `dbt_valid_to = NULL` (every patient is current — this is the initial load)
- All rows share the same `dbt_valid_from` timestamp

Task 3 — Simulate a change and observe SCD2 in action

These steps must be run in order. Skipping Step 2 is the most common mistake.

Step 1 — Simulate a data change

Snowflake: Run an UPDATE on the `HUBSPOT_RAW.contacts` table:

```
UPDATE DBT_TRAINING.HUBSPOT_RAW.contacts
SET email = 'new.email@example.com', updated_at = '2024-05-11 08:00:00'
WHERE id = 3;
```

DuckDB local testing: Edit `seeds/raw_contacts.csv` — find `contact_id = 3`. Change their `email` to any new address. Bump `updated_at` by one day (e.g., from `2024-05-07 16:45:00` to `2024-05-08 16:45:00`).

Step 2 — Reload the source data (DuckDB only — Snowflake skip this)

```
dbt seed --target test --full-refresh --select raw_contacts
```

This reloads the changed CSV into DuckDB. Without this, the snapshot sees no changes.

Step 3 — Run `dbt snapshot`

```
# Snowflake:
dbt snapshot
```

```
# DuckDB:
dbt snapshot --target test
```

dbt detects the change in `contact_id = 3` (the `updated_at` value is newer than what the snapshot stored) and inserts a new SCD2 row.

Step 4 — Query the snapshot table

```
SELECT
    contact_id,
    email,
    dbt_valid_from,
    dbt_valid_to
FROM snapshots.snap_patients
WHERE contact_id = 3
ORDER BY dbt_valid_from;
```

Expected: Exactly **two rows** for contact 3:

- Row 1: original email address, `dbt_valid_to` set to the change timestamp (non-NULL)
- Row 2: new email address, `dbt_valid_to` = NULL (current row)

All other contacts should still have exactly one row.

Verify:

```
SELECT contact_id, COUNT(*) AS row_count
FROM snapshots.snap_patients
GROUP BY contact_id
ORDER BY row_count DESC;
```

Only `contact_id = 3` should have `row_count = 2`.

Bonus — Answer a historical question

Write a SQL query that answers: "What email address did patient 3 have on 2024-06-01?"

Use only the snapshot table. Do not look at the source data.

Requirements:

- Returns exactly one row
- Uses `dbt_valid_from` and `dbt_valid_to` to find the active version on that date
- Handles the current row correctly (hint: `dbt_valid_to` = NULL means "still active")

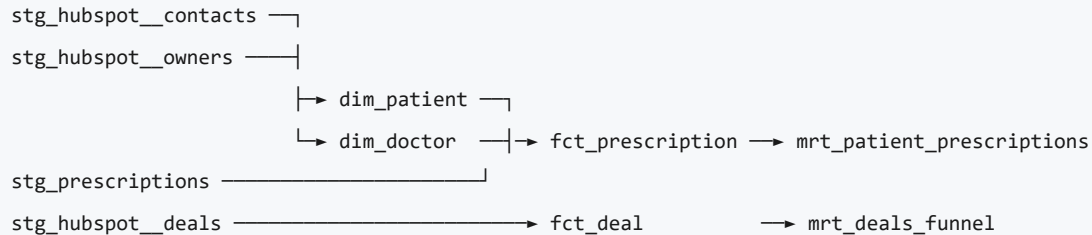
► Expected solution

Module 11 — Selectors, Tags, and Running Subsets

Duration: 45 min

Context: You are working in the exercise project. Write the exact `dbt` command for each scenario below. You do not need to run every command — but try each one and check the output of `dbt ls` to confirm you selected the right models.

For reference, here is the relevant subset of the exercise project's DAG:



Scenario 1 — Rebuild only the Silver fact models and their tests

"The Silver fact tables need a clean rebuild after a source schema change. Run only the Silver facts with their tests."

Write the command.

► Answer

Preview with `dbt ls` :

```
dbt ls --select silver.facts.*
```

Scenario 2 — Run `fct_prescription` and everything it depends on

"Something is wrong with `fct_prescription`. You want to rebuild it from scratch, including all the models it reads from."

Write the command.

► Answer

Preview with `dbt ls` :

```
dbt ls --select +fct_prescription
```

Scenario 3 — Run `mrt_deals_funnel` and all models downstream of it

"You want to see what `mrt_deals_funnel` produces and whether anything reads from it. Run it and everything that depends on it."

Write the command.

► Answer

Preview with `dbt ls` :

```
dbt ls --select mrt_deals_funnel+
```

Scenario 4 — Run all daily-tagged models but skip Gold marts

"The nightly job should rebuild Silver facts but not the Gold marts (which run separately on Sunday). Write the command."

Write the command.

► Answer

Preview with `dbt ls` :

```
dbt ls --select tag:daily --exclude gold.*
```

Scenario 5 — After a partial failure, rerun only the models that errored

"Last night's `dbt build` failed on 3 models. The other 15 ran cleanly. Rerun only the failures."

Write the command.

► Answer

Bonus — Add a tag and write the selection command

1. Open `dbt_project.yml` and add a `finance` tag to `fct_deal` and `mrt_deals_funnel`.

You can add this in `dbt_project.yml` (folder- or model-level using `+tags:`) or in each model's `schema.yml` config block. Both work.

1. Write a command to run only finance-tagged models with all their tests.

► Answer

Module 12 — CI/CD and Slim CI

Duration: 90 min

Context: This module has no dbt project exercise tasks — no `dbt build` commands needed. The exercise is a code review of a broken GitHub Actions YAML. Work through the tasks on paper or in a text editor.

Task 1 — Fix the broken CI config

The workflow file below has **three bugs**. For each bug:

1. Identify what is wrong
2. Explain why it is a problem (one sentence)
3. Write the corrected line(s)

```
name: dbt CI

on:
  pull_request:
    branches: [main]

jobs:
  slim_ci:
    runs-on: ubuntu-latest

    env:
      DBT_TARGET: ci
      SNOWFLAKE_ACCOUNT: ${ secrets.SNOWFLAKE_ACCOUNT }}
      SNOWFLAKE_USER:    ${ secrets.SNOWFLAKE_USER }}
      SNOWFLAKE_PASSWORD: "Snowflake_Prod_2024_v3"
      SNOWFLAKE_DATABASE: ${ secrets.SNOWFLAKE_DATABASE }}
      SNOWFLAKE_WAREHOUSE: ${ secrets.SNOWFLAKE_WAREHOUSE }}
      SNOWFLAKE_ROLE:    ${ secrets.SNOWFLAKE_ROLE }}
      DBT_SCHEMA:        DBT_CI_${ github.run_id }}

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install dbt
```

```

run: pip install dbt-snowflake==1.8.4

- name: Download production manifest
  run: |
    mkdir -p ./prod-artifacts
    aws s3 cp s3://bloomwell-dbt-artifacts/prod/manifest.json ./prod-artifacts/manifest.json
  env:
    AWS_ACCESS_KEY_ID:      ${ secrets.AWS_ACCESS_KEY_ID }
    AWS_SECRET_ACCESS_KEY:  ${ secrets.AWS_SECRET_ACCESS_KEY }
    AWS_REGION:             eu-central-1

- name: dbt deps
  run: dbt deps

- name: dbt build - slim CI
  run: |
    dbt run \
      --select state:modified+ \
      --defer \
      --state ./target \
      --target ci

```

► The three bugs and fixes

Task 2 — Write the CI `--select` command

You are opening a PR that modifies only `fct_deal.sql`. The model graph looks like this:

```

stg_hubspot__deals → fct_deal → mart_revenue
                    → mart_pipeline_summary

```

Write the exact `dbt build` command you would use in CI to:

- Build only the changed model and its downstream dependents
- Use production data for upstream staging models (do not rebuild `stg_hubspot__deals`)
- Compare against the production manifest stored at `./prod-artifacts`

► Answer

Task 3 — Explain `--defer`

Answer each question in exactly one sentence.

1. What does `--defer` do when dbt encounters an unmodified upstream model?
2. Why does this matter for CI compute cost?
3. What would happen without `--defer` if only one Gold mart changed?

Tier 2 Complete

You have completed all Tier 2 exercises. You can now:

- Load and reference seeds in models with `{{ ref() }}`
- Use variables to remove hardcoded constants from SQL and override them at runtime
- Write macros for repeated structural patterns and call them in models
- Use `dbt_utils.generate_surrogate_key` to create stable surrogate keys
- Create dbt snapshots with the `timestamp` strategy to track SCD2 history
- Simulate data changes end-to-end: edit a seed CSV, reload with `dbt seed`, and observe SCD2 rows
- Write precise selection commands using graph operators (`+model` , `model+`), tags, and `result:error`
- Identify and fix the three most common CI configuration bugs

Next: Tier 3 — Production and Advanced Patterns covers advanced testing, complex incremental strategies, the `scd2_merge` macro deep dive, and model contracts.